L37: Simulation-Based Optimization

Simulation Without a Black Box

Outline

The SimOpt Problem

Why Gradient Methods Fail

Ranking and Selection as a Special Case

Random and Grid Search

Discrete SimOpt Algorithms

Budget Allocation

Convergence

Key Takeaways

The Simulation Optimization Problem

Formal Statement

$$\min_{x \in \mathcal{X}} \ \mathbb{E}_{\omega}\left[f(x, \omega)\right]$$

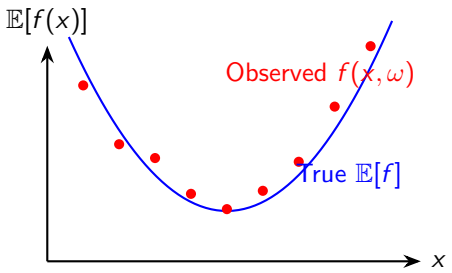
- x: decision vector (staffing, reorder point, routing rule)
- ω: simulation randomness (seed / sample path)
- $\mathcal{X}$: feasible set (integer, combinatorial, or continuous)
- $f(x, \omega)$: observed performance in one replication

Key Difficulty

We can never evaluate $\mathbb{E}[f(x, \omega)]$ exactly — we observe noisy estimates $\bar{f}(x) = \frac{1}{n}\sum_i f(x, \omega_i)$. Every evaluation costs simulation time; the budget is finite.

Example: choose (s, S) inventory policy to minimise expected total cost per period.

Why Gradient-Based Methods Fail on Simulation



Problems with classical gradient methods:

- Finite differences amplify noise: $\hat{\nabla} \approx \frac{f(x+h,\omega_1)-f(x,\omega_2)}{h}$ is dominated by noise for small h
- Gradient is undefined for discrete $\mathcal{X}$
- Local optima in multi-modal surfaces

Alternatives needed: sample-average approximation, ranking and selection, metaheuristics, surrogate models.

Ranking and Selection: Finite Candidate Set

R&S Setup

k candidate designs $x_1, \ldots, x_k$ (e.g., $k = 12$ staffing levels). Goal: identify the best with high probability $P^* \geq 0.95$.

Indifference zone: tolerate errors $\leq \delta$ in performance.

KN procedure (Kim & Nelson):

1. Take n_0 pilot replications of all k designs.
2. Eliminate designs that are clearly inferior (outside δ of best).
3. Continue sampling survivors until one remains.

```python
from simdes.analysis import
↪  ranking_selection
result = ranking_selection(
    simulator=MM1Queue,
    designs=[
        {"c":1}, {"c":2}, {"c":3}
    ],
    kpi="Wq",
    alpha=0.05,
    delta=0.5,    # indifference zone
    n0=10,
    seed=42
)
print(result["best_design"])
```

Baseline Methods: Grid Search and Random Search

Grid Search

Enumerate all x on a regular grid. Run n replications per point. Select the grid point with lowest $\bar{f}$.

Pro: simple, fully parallel.
Con: curse of dimensionality — m points per dimension, d dimensions $\Rightarrow m^d$ evaluations.

Random Search

Sample x uniformly from $\mathcal{X}$, evaluate, keep best.

Pro: no grid required; anytime algorithm.
Con: does not exploit response surface structure.

When to use: warm-start for more sophisticated methods; baseline to beat in any comparison.

Warning

Reporting the best observed design without a confidence interval on $\mathbb{E}[f(x^*)]$ is insufficient — the optimistic bias from multiple comparisons can be severe.

Algorithms for Discrete Decision Spaces

COMPASS (Xu et al., 2010)

- Maintains a sample set around the current best
- Converges to the true optimum with probability 1 under mild conditions
- Efficient for small-to-medium integer problems

Simulated Annealing (SA)

- Accepts worse solutions with probability $e^{-\Delta/T}$
- Temperature T decreases on schedule (cooling)
- Escapes local optima; no gradient required

```
# Pseudocode for SA on discrete x
x = x_init; T = T_init
for step in range(budget):
    x_new = neighbor(x)
    delta = f_hat(x_new) - f_hat(x)
    if delta < 0 or rand() < exp(-delta/T):
        x = x_new
    T *= cooling_rate
```

Cooling Schedule

Geometric: $T_k = T_0 \cdot \alpha^k$ with $\alpha \in [0.85, 0.99]$.
Too fast $\Rightarrow$ premature convergence.

Budget Allocation: Replications per Candidate

Total budget: B replications shared among k designs.

Naive equal allocation: $n_i = B/k$ per design.

OCBA (Optimal Computing Budget Allocation):

$$\frac{n_i}{n_j} = \left(\frac{\sigma_i/\delta_i}{\sigma_j/\delta_j}\right)^2, \quad \delta_i = |\bar{f}_i - \bar{f}^*|$$

Allocate more replications to designs that are close to the current best *and* highly variable.

Intuition

We need more samples to distinguish two designs that are close together than two designs that are far apart. OCBA formalises this using variance and distance from the current best.

Pitfall

Equal allocation wastes budget on clearly inferior designs. Even two rounds of OCBA reallocation can cut required budget by 30–50%.

Convergence: When to Stop Searching

Stopping criteria:

1. **Budget exhausted** — most common in practice.
2. **No improvement for M consecutive iterations** — stagnation check.
3. **Statistical indistinguishability** — current best and all neighbors overlap in their CIs.
4. **Predicted improvement below threshold** — used in Bayesian optimization (EI $< \epsilon$).

Final Validation

After stopping:

Run $n_{\text{final}} \gg n_{\text{search}}$ replications on the declared best design and confirm its CI does not overlap with the runner-up.

Never report the search-phase estimate as the final result.

Key Takeaways

L37 Summary

1. SimOpt minimises $\mathbb{E}[f(x, \omega)]$ over a feasible set using noisy function evaluations.
2. Gradient methods fail for noisy, discrete, or multi-modal problems — use R&S, SA, COMPASS, or surrogate methods.
3. R&S is the right tool when the candidate set is small and enumerable.
4. OCBA allocates budget intelligently — spend more replications on close competitors, not clearly inferior designs.
5. Always re-validate the declared optimum with fresh replications before reporting.

Next: L38 — Kriging surrogate models and Bayesian optimization.